Reduzindo a complexidade com Estruturas de Dados

SCC5900 - Proj. Algoritmos - 2026

Motivação

- Seja um vetor V de n inteiros pequenos e considere a tarefa de encontrar o k_{th} menor elemento deste vetor.
  - Qual seria uma solução força bruta?
  - E qual seria a complexidade desta solução?
- Você tem uma solução 'divisão e conquista' para isso, de menor complexidade?
- O que fizemos acima foi alterar a complexidade da solução, escolhendo um paradigma de programação diferente:
  - Saímos do força bruta para uma solução divisão e conquista e o ganho foi considerável

Motivação

- Uma estratégia muitas vezes negligenciada é investir em uma estrutura de dados sem necessariamente alterar o paradigma de programação

- Selecionando um par de inteiros em um vetor
  - Dado um vetor de n inteiros, verifique se há elementos repetidos nele.
  - Existe uma solução muito simples de complexidade $O(n^2)$.
  - Você consegue pensar em uma que tenha solução O (n log(n)) ????

Solução 1: for aninhado

```
1  #include <iostream>
2  using namespace std;
3
4  bool hasDuplicates(int vet[], int n) {
5      for (int i = 0; i < n-1; i++) {
6          for (int j = i+1; j < n; j++) {
7              if (vet[i] == vet[j]) {
8                  return true;
9              }
10         }
11     }
12     return false;
13 }
14
15 int main() {
16     int vet[] = {3, 5, 8, 5, 12, 15};
17     int n = sizeof(vet)/sizeof(vet[0]);
18     if (hasDuplicates(vet, n))
19         cout << "Ha elementos repetidos" << endl;
20     else cout << "Nao ha elementos repetidos" << endl;
21
22     return 0;
23 }
```

Solução 2: por que usar *set* leva a complexida menor?

```
#include <iostream>
#include <set>
using namespace std;

bool hasDuplicates(int vet[], int n) {
    set<int> seen;

    for (int i = 0; i < n; i++) {
        if (seen.find(vet[i]) != seen.end()) {
            return true;
        }
        seen.insert(vet[i]);
    }
    return false;
}

int main() {
    int vet[] = {3, 5, 8, 5, 12, 15};
    int n = sizeof(vet)/sizeof(vet[0]);
    if (hasDuplicates(vet, n))
        cout << "Ha elementos repetidos" << endl;
    else  cout << "Nao ha elementos repetidos" << endl;

    return 0;
}
```

Encontrando elementos repetidos

- Uso de set()
  - Elimina o loop aninhado!
  - Operações de inserção, remoção e busca de ordem O(log(n)). Por que?
    - Por que a ED set é implementada como uma árvore binária balanceada (red/black tree, parecida com AVL).
  - As operações de inserção, remoção e busca são de ordem O(log(n))
- Claramente a complexidade do código agora é O(n log(n)).
  - isso é um ganho extremamente significativo, especialmente se n for muito grande !

Complexidades Notáveis

- Complexidade constante - O(1): quando a eficiência de um algoritmo não depende de nenhum valor variável

```
1  int x = 10;
2  int y = 50;
3  cout << x + y << endl;
```

Complexidades Notáveis

- Complexidade em um laço de repetição O(n):
  - obtida a partir da quantidade de repetições.

```
1  for (int i = 0; i < n; i++) {
2      ...
3  }
```

- Se dentro do laço houver alguma operação de complexidade não constante, deve levá-la em consideração. Seja essa complexidade = g(n)
  - Portanto, a complexidade final será O(n * g(n))

Complexidades Notáveis

- Complexidade em um laço de repetição: para frações de código independente, a complexidade é a soma das complexidades de cada um
  - obtida a partir da quantidade de repetições.

```
for (int i = 0; i < n; i++) {
    ...
}

for (int i = 0; i < m; i++) {
    ...
}
```

Complexidades Notáveis

- Complexidade em um laço de repetição: Se os laços forem aninhados, a complexidade final é a multiplicação dos seus tamanhos

```
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        ...
    }
}
```

- A complexidade acima é $O(n^2)$

Complexidades Notáveis

- Complexidade em um laço de repetição : um pouco mais complexa, mas bastante comum. Veja o exemplo abaixo:

```
1  for (int i = 0; i < n; i++) {
2      for (int j = i; j < n; j += i) {
3          ...
4      }
5  }
```

- Embora parece quadrática, é sub-quadrática = O(n * log(n)). Uma série harmônica [Soma (i=0..n) 1/i] tem complexidade O(log(n)).

Estimando recursos necessários

- Memória: ver quantos bytes serão usados. Fique também atento aos limites de stack e heap, variáveis globais, etc.
- A análise assintótica pode (notação big O) não possui uma relação única e direta com o tempo de execução do seu código. POr exemplo, se a entrada for muito pequena.
- O tempo de execução muitas vezes depende de: poder de processamento, flags de compilação, e da **linguagem de programação**.
- Operações de ponto flutuante tb são um fator a considerar.

Estimando recursos necessários

- Uma regra útil: 10^8 operações em C++ >>> 1 segundo !
- Para $n = 10^5$, um código de complexidade $O(n^2)$ levaria cerca de 100 segundos.

Estimando recursos necessários

Complexidade	n para executar em 1s	Exemplo de algoritmo
$\mathcal{O}(\log n)$	$2^{100000000}$	Busca binária
$\mathcal{O}(\sqrt{n})$	10^{16}	Encontrar divisores de um número inteiro
$\mathcal{O}(n)$	10^{8}	Busca linear em um vetor
$\mathcal{O}(n \log n)$	10^{6}	Ordenar um vetor
$\mathcal{O}(n\sqrt{n})$	10^{5}	Algoritmo de Mo
$\mathcal{O}(n^2)$	10^{4}	Busca binária
$\mathcal{O}(n^3)$	500	Floyd Warshall
$\mathcal{O}(2^n)$	26	Máscaras binárias (*bitmasks*)
$\mathcal{O}(n!)$	12	Encontrar permutações de tamanho n